COMP9123
Lecture 1
Unit Overview

Lecturer: Dr Armin Chitizadeh

Version: 1.1

THE UNIVERSITY OF SYDNEY

Inspired by Hazem El-Alfy 2024 slides

CRICOS 00026A TEQSA PRV12057

We recognise and pay respect to the Elders and communities – past, present, and emerging – of the lands that the University of Sydney's campuses stand on. For thousands of years they have shared and exchanged knowledges across innumerable generations for the benefit of all.

THE UNIVERSITY OF SYDNEY

COMP9123 – Data Structures and Algorithms

- This unit is your formal introduction to algorithms and data structures, which will lay the foundations for you to come up with your own algorithms and data structures and be confident of their correctness.

Our Goal

- We would like you to develop an **analytical mindset** with which you could approach a real-world problem. You would be able to understand suitable data structures and algorithms to address this problem and then design and test your solutions with confidence.

Agenda

- Unit arrangement
- Expectations
- Assessment
    - Unit details
    - Policies
- Workplace Health and Safety
- Assistance
- Advice

Agenda

Places

- Lecture:
  - Wednesdays 7-9 PM
  - Will be shared live and recorded on echo360 (canvas)
  - We try some ways for online people to be more engaged

- Tutorial:
  - Check your timetable
  - Starts in Week1
  - Not recorded

- Things to keep in mind:
  - Please go to your tutorial (All of them are packed)
  - Be prepared

People

- Lecturer and Unit Coordinator:
  - Dr. Armin Chitizadeh (armin.chitizadeh@sydney.edu.au)

- TAs:
  - ☹

- Tutors:
  - Yihao Dong – 8
  - Abbey Lin – 7
  - Lindsey Deryckere - 1
  - Islam Alzoubi - 15, 16
  - Leon Kennedy 11, 12
  - Maryam Hashemi 9, 10, 13
  - Niousha Nazemi 2
  - Pasindu Marasinghe 3, 4
  - Thilina Lokuruge 5, 6, 14

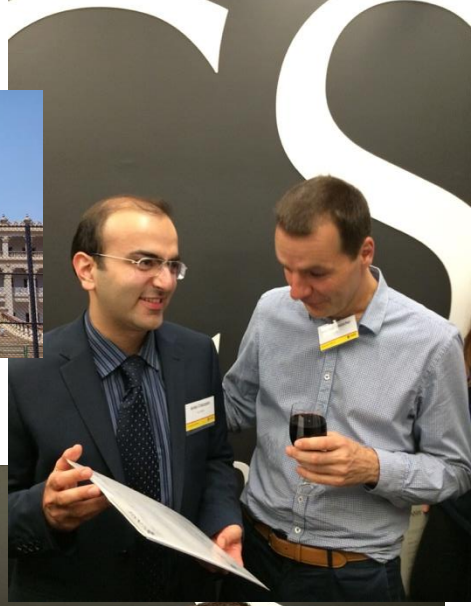
Fully anonymous feedback form for COMP9123 students – Fill out form

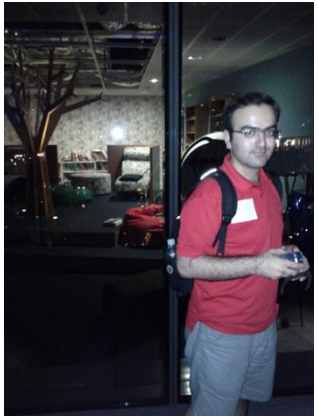
Let’s know each other!

Mentimeter

https://www.menti.com/al8ks37ny3ca

Why I chose





Microsoft

Resources

- Canvas:
  - Login using Unikey and password
  - Link to Units website: https://sydney.edu.au/units/
    - Official Schedule, list of learning outcomes, etc
  - Lecture videos
  - Weekly quizzes
  - Ed:
    - Discussion forum
    - Lecture slides – for most updated version, download right before the lecture (look for orange colour)
    - Tutorial instruction and solutions
    - Assignment instructions and solutions
    - Extra resources, including programming exercises

Reference book

- “Algorithm Design and Applications”, by Goodrich and Tamassia: published by Wiley
    - It is not required for purchase, but you may find it useful to guide your exploration of the idea
- This and other relevant works can be found in the university library

- Link to “Algorithms”, by Jeff Erickson can be found on Ed
- There is also a growing list of additional resources available on this Ed Post:
    - (To be shared next week)

Mid-semester Revision Session During Stuvac
(To be confirmed and to get funded)

Expectations from students

- Students attend scheduled classes, and devote an extra 6 hours per week
    - Doing assessments
    - Preparing and reviewing for classes
    - Revising and integrating the ideas
    - Practice and self-assess

- Students are responsible learners
    - Participate in classes, constructively
    - Respect for one another: criticize ideas, not people
    - Humility: none of us knows it all

- Notify teaching staff whenever there are difficulties

- Please attend your tutorials

Teaching Team Expectations

- Deliver a high quality teaching
- Listen to student feedback and concern
- Making sure tutorials are of high quality

- Further suggestions?

Assessment

- 3 Assignment
  - Assignment1 (Week4 –)
  - Assignment2 (Week9 –)
  - Assignment2 (Week13 –)

- 10 Quizzes
  - Every week from week3
  - 10% of final mark
  - Equally weighted

  - Exam
    - 50% of final mark
    - In-person / supervised
    - University policy: You must get
      - An oval mark of 50% or more
      - At least 40% of the mark available in the exam, in order to pass the unit

Tutorials

- We will post a tutorial sheet with exercises covering that week's material
- To get the most out of tutorial, try to solve as many problems as you can **before the tutorial**
- After all tutorials of a given week are over, we will post solutions to the tutorials on the following Monday

Communication

- Email:
    - Only email if it is a private matter
- Ed Discussion:
    - Please post public questions
    - Questions that can be asked and answered in few sentences:
        - Lecture materials
        - Homework
        - Assignments
        - Logistic related to classes
- Asking directly:
    - Asking your tutor
    - Consultations
    - After the lecture

Ed etiquette

- Only Ask questions that can be asked and can be answered within few lines
- When asking questions:

    - Use Ed’s search function to see if it’s been answered before
    - When asking a new question:
        - Add a descriptive title
        - Select the correct tag, e.g. questions about Quiz4 should be tagged with Quizzes -> Q4
        - Describe what you’ve already tried to solve the problem
        - Don’t just take a screenshot and say “what does this mean?”
- If you figured out your own, share your answer to others
- If you can, please answer questions

Ed expectations

- Do not expect any response outside of working hours (Mon-Fri 9am – 5pm) and on weekends and public holidays
- Please allow up to 48 hours for your question to be answered before reposting.
- If you do not receive a response even after reposting, please bring to my attention vai email or before/ after the lecture
- Support each other

- There is no TA helping with answering questions

Simple Extension

- If your performance on assessments is affected by illness or misadventure
    - Simple extension – The University of Sydney

Simple Extension vote

Everyone will get simple extension.
If you want to get the mark sooner, you need to fill a form.

Special Consideration (University Policy)

- If your performance on assessments is affected by illness or misadventure
- Follow proper bureaucratic procedures
    - Have professional practitioner sign special Usyd form
    - Submit application for special consideration online, upload scans
    - Note you have only a quite short deadline for applying
    - http://sydney.edu.au/current_students/special_consideration/
- Also, notify coordinator by email as soon as anything begins to go wrong
- There is a similar process if you need special arrangements e.g. for religious observance, military service, representative sports

Late submission

- Suppose you hand in work after the (simple extension) deadline:
- If you have not been granted special consideration or arrangements
    - A penalty of 5% of the available marks will be taken, per day (or part) late for the first 2 days, after that your mark becomes 0
- E.g. your work would scored 60% and is 1 hour late
    - You get 55%
- E.g your work would have scored 70% and is 1 day 23 hours late
    - You get 60%
- Warning: submission sites get very slow near deadlines
- Submit early; you can resubmit as many times as you wish before the deadline

Academic Integrity (University policy)

- "The University of Sydney is unequivocally opposed to, and intolerant of, plagiarism and academic dishonesty.
    - Academic dishonesty means seeking to obtain or obtaining academic advantage for oneself or for others (including in the assessment or publication of work) by dishonest or unfair means.
    - Plagiarism means presenting another person's work as one's own work by presenting, copying or reproducing it without appropriate acknowledgement of the source." [from site below]

http://sydney.edu.au/elearning/student/EI/index.shtml

- Submitted work is compared against other work (from students, the internet, etc.)
    - TurnitIn for textual tasks (*your document needs to be scannable, i.e., answers stored as text and no handwriting*), other systems for code
- Penalties for academic dishonesty or plagiarism can be sever and can delay the release of your result at the end of the semester

Academic Integrity Practice

School of Computer Science

Understanding your academic integrity responsibilities

Academic integrity refers to behaving honestly, ethically and responsibly in relation to all elements of your study at the University, including assessments.

It's important that you:

- **Always submit your own work**, sit your own tests, and take your own examinations.
- **Acknowledge any contributions** in your assignment which are not your original thoughts, ideas or words.
- **Complete the Academic Honesty Education Module (AHEM)**, which all new students must complete by census date (31 March 2025).
- **Listen to your coordinator's instructions** on whether digital tools, including grammar checkers and generative AI, are allowed in assessments.

Complete the AHEM

Using generative AI

Generative AI tools, such as Microsoft Copilot Chat, can create digital content, including text, images, and video. Different guidelines for use of generative AI apply to learning and assessment:

Generative AI for general learning

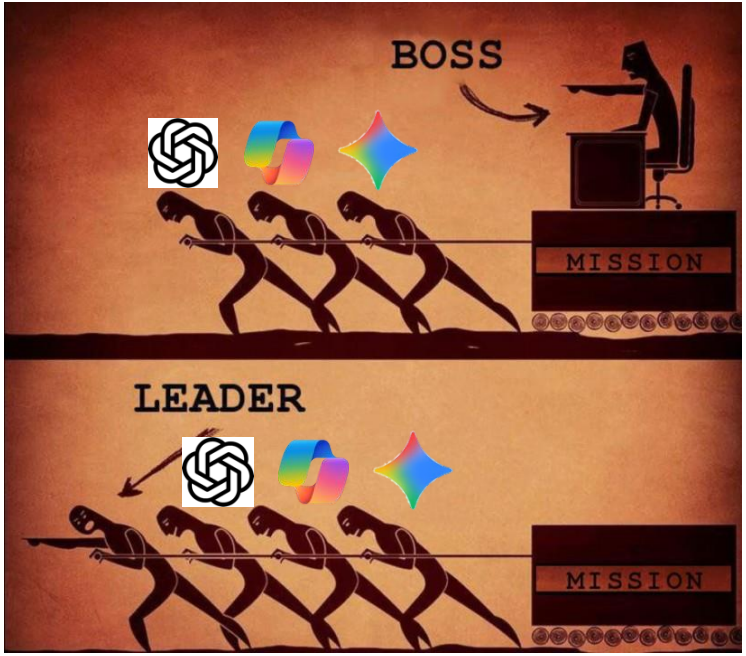
- You can use generative AI to help you learn. For example, it can explain things, apply knowledge, plan your study and make practice questions. Use our student co-designed AI resource so you are aware of its strengths and limitations.

Generative AI for assessment

- *Exams and in-semester tests:* you **cannot** use generative AI in supervised exams or in-semester tests, unless your unit coordinator expressly allows it.
- *Other assessments:* you **can** use generative AI, unless your unit coordinator expressly prohibits it.
- Always acknowledge your AI use. Failing to do so, or using AI when not permitted, is an academic integrity breach.
- Each unit will have different rules. Look for the AI icons ✦ in the assessment table in your unit outlines to see if AI is allowed.

AI resource made by students:
https://bit.ly/students-ai

Why we need to learn anything in the era of GenAI?



Which generative AI tools are best to use?

The University provides **free** access to Microsoft Copilot Chat to accelerate your learning and help you complete those assessments where use of generative AI is allowed.

Why use Microsoft Copilot Chat?

- It is a University-supported, industry standard, GPT-4 powered tool
- It keeps your data secure and protects your privacy
- Log in with your UniKey using Okta so that you are using Copilot in protected mode, which introduces necessary guardrails to protect you within the system

Why not use other tools like ChatGPT and DeepSeek?

- Don't risk your privacy or the security of your personal and assessment data by using a free or emerging tool like ChatGPT and DeepSeek

Powerful and secure
https://copilot.microsoft.com/

Completing your mandatory Canvas Modules

We've developed a series of Canvas modules to help you understand what to expect from the University and what is expected from you. They include:

- Respect@Sydney, designed to give you an understanding of consent, boundaries and respectful relationships *(mandatory for commencing students)*.
- Academic Honesty Education Module *(mandatory for commencing students)*.
- Engaging with Civility, a module where you can unpack the values and expectations outlined in our Student Charter, including information about penalties for breaching our codes of conduct and displaying threatening, discriminatory or racist behaviour.
- Anti-Slavery Awareness Module.

Many of these modules are mandatory for commencing students and your results may be withheld if you do not complete them.

Student Support

- Visit the Student life, wellbeing and support webpage to find out
- about the student services, resources and events available to
- support you while you study:
    - Health and wellbeing
    - Academic Support
    - Personal support
    - Getting connected

- Questions about getting started this semester? Come visit us at a Welcome Hub

Anderson Stuart
Welcome Hub

Carslaw West
Welcome Hub

Safer Communities Office

- Support and case management for people who have experienced sexual misconduct, domestic/family violence, bullying/harassment or issues relating to modern slavery.
- Contact the team
  - 8:30 am to 5:30 pm Monday to Friday, Sydney local time
  - phone: +61 2 8627 6808
  - email: safer-communities.officer@sydney.edu.au
  - campus: Level 5, Jane Foss Russell building, City Road, Darlington Campus
- Make a report
  - Visit the website to make a complaint or disclosure of sexual misconduct to the University.

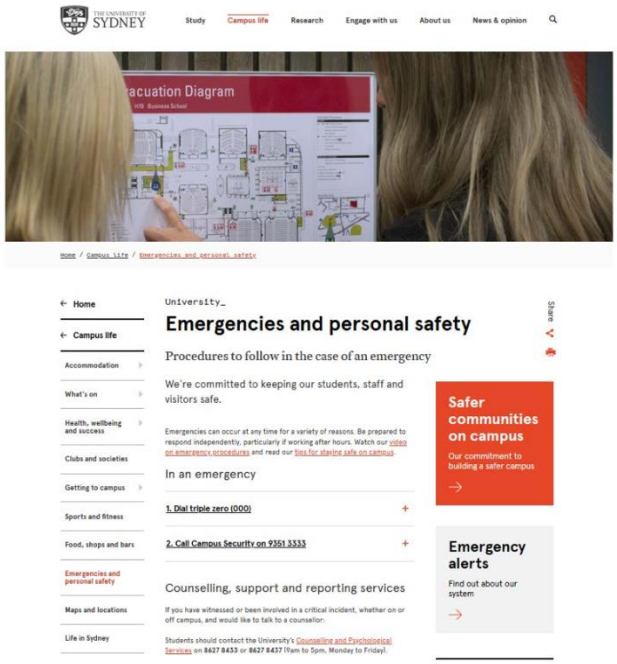
> Make a report link <

General Housekeeping – Use of Labs

- Keep work area clean and orderly
- Remove trip hazards around desk area
- No food and drink near machines
- No smoking permitted within University buildings
- Do not unplug or move equipment without permission

Emergencies – Be prepared

https://sydney.edu.au/campus-life/safety-security.html



Assistance

- There are a wide range of support services available for
- students: https://sydney.edu.au/campus-life/health-
- wellbeing-success.html
- Please make contact, and get help
- You are not required to tell anyone else about this
- If you are willing to inform the unit coordinator, they may be able to work with other support to reduce the impact on this unit
    - E.g. provide advice on which tasks are most significant

Disability Service

Do you have a disability?

- You may not think of yourself as having a 'disability' but the definition under the **Disability Discrimination Act** is broad and includes temporary or chronic medical conditions, physical or sensory disabilities, psychological conditions and learning disabilities.
- The types of disabilities we see include:
- anxiety, arthritis, asthma, asperger's disorder, ADHD, bipolar disorder, broken bones, cancer, cerebral palsy, chronic fatigue syndrome, crohn's disease, cystic fibrosis, depression, diabetes, dyslexia, epilepsy, hearing impairment, learning disability, mobility impairment, multiple sclerosis, post traumatic stress, schizophrenia , vision impairment, and much more.
- Students needing assistance must register with Disability Services –
  - it is advisable to do this as early as possible.

- https://www.sydney.edu.au/students/health-wellbeing/inclusion-and-disability.html

Do you have a disability?

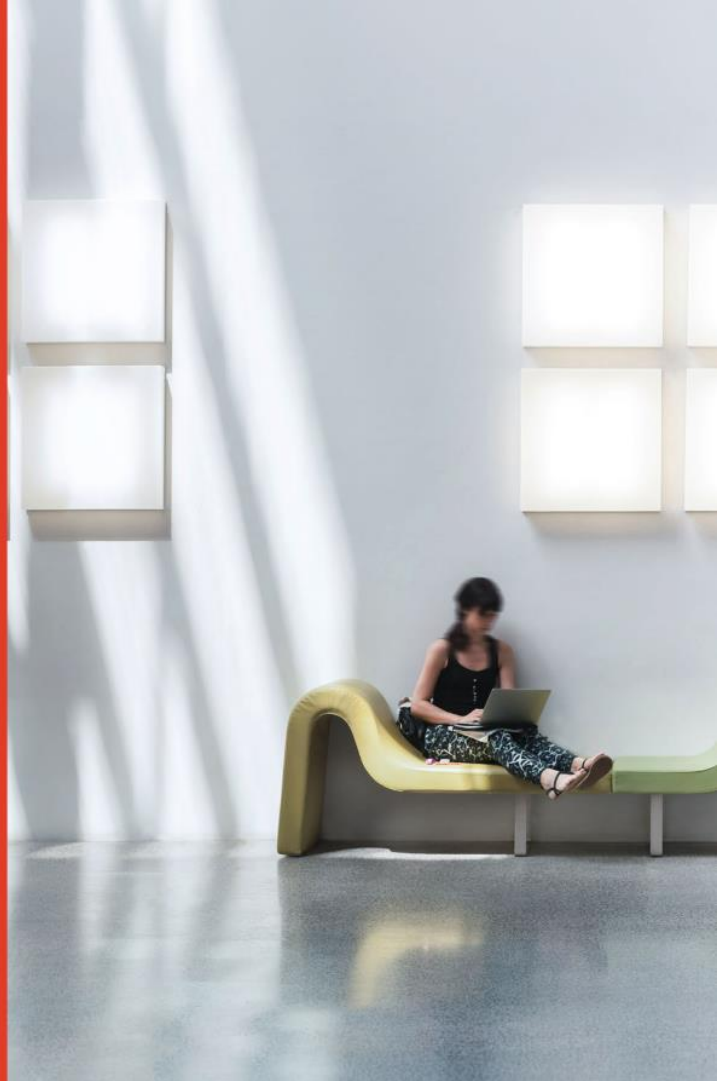
You may not think of yourself as having a 'disability' but the definition under the **Disability Discrimination Act (1992)** is broad and includes temporary or chronic medical conditions, physical or sensory disabilities, psychological conditions and learning disabilities.

The types of disabilities we see include:
Anxiety // Arthritis // Asthma // Autism // ADHD
Bipolar disorder // Broken bones // Cancer
Cerebral palsy // Chronic fatigue syndrome
Crohn's disease // Cystic fibrosis // Depression
Diabetes // Dyslexia // Epilepsy // Hearing impairment //
Learning disability // Mobility impairment // Multiple sclerosis // Post-traumatic stress // Schizophrenia //
Vision impairment
and much more.

Students needing assistance must register with Disability Services. It is advisable to do this as early as possible. Please contact us or review our website to find out more.

THE UNIVERSITY OF SYDNEY

Disability Services Office
sydney.edu.au/disability
02-8627-8422



Other Support

- Learning support
  - http://sydney.edu.au/study/academic-support/learning-support.html

- International students
  - http://sydney.edu.au/study/academic-support/support-for-international-students.html

- Aboriginal and Torres Strait Islanders
  - http://sydney.edu.au/study/academic-support/aboriginal-and-torres-strait-islander-support.html

- Student organization (can represent you in academic appeals etc)
  - http://srcusyd.net.au/ or http://www.supra.net.au/

- Please make contact, and get help

- You are not required to tell anyone else about this

- If you are willing to inform the unit coordinator, they may be able to work with other support to reduce the impact on this unit
  - eg provide advice on which tasks are most significant

Advice

- MetaCognition
  - Pay attention to the learning outcomes in Canvas
  - Self-check that you are achieving each one
  - Think how each assessment task relates to these

- Time management
  - Watch the due dates
  - Start work early, submit early
  - Keep a health balance

- Networking and community-formation
  - Make friends and discuss idea with them
  - Know your tutor, lecturer, coordinator
  - Keep them informed, especially if you fall behind
    - Don’t wait to get help

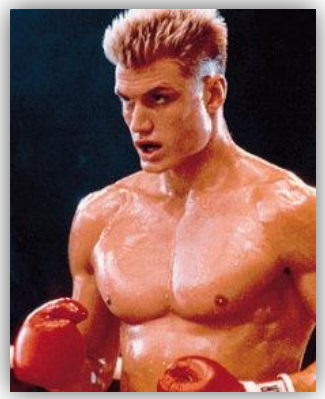
- Enjoy the journey!

We are here to help

You

COMP9123 teaching team

Life



We are here to help

COMP9123: Data Structure & Algorithm

Introduction

COMP9123 – Data Structure & Algorithms

- This unit is your formal introduction to algorithms and data structures, which will lay the foundations for you to come up with your own algorithms and data structures and be confident of their correctness.

Out Goal

- We would like you to develop an **analytical mindset** with which you could approach a real-world problem. You would be able to understand suitable data structures and algorithms to address this problem and then design and test your solutions with confidence.

Algorithms

Algorithms

- An **algorithm** is a step-by-step procedure or set of rules to solve a problem or perform a task.

Key Characteristics:

- **Finite**: It has a clear beginning and end
- **Precise**: Each step is well-defined
- **Effective**: It produces the desire result

Algorithms

- Algorithms are everywhere: in cooking, navigating, and technology
- They help us break down complex problems into manageable steps
- We'll explore more examples and techniques throughout this unit

Real World Examples of Algorithms

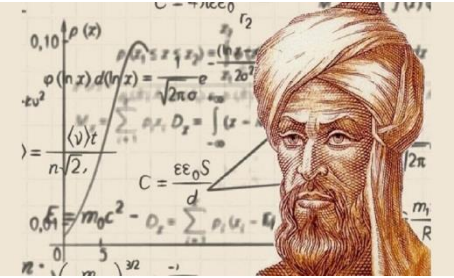
Commercial contract
How to put elephant into the fridge
Sorting

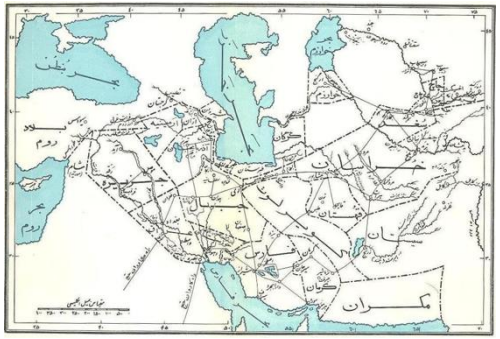
“Algoritmi”

- The word "algorithm" comes from the name of a 9th-century Persian mathematician: Al-Khwarizmi (Muḥammad ibn Mūsā al-Khwārizmī).
    - The Latinized form of his name, *Algoritmi*, evolved into the term "algorithm.”

- Al-Khwarizmi’s Key Contributions:
    - Wrote a book on solving mathematical problems using step-by-step procedures.
    - His work laid the foundation for modern algebra and computational methods.

- Fun Fact:
    - Al-Khwarizmi's book also introduced the concept of zero and positional number systems, which are vital to modern computing.





What are some examples of everyday algorithms?

Common Types of Algorhms

- Sorting
- Searching
- Graph
- Optimization

A simple Example Making a Sandwich

- Problem: How do you make a sandwich?
- `Algorithm:`

Another Example: Finding the Largest Number

- Problem: Given three numbers, find the largest.

```
Algorithm:
```

Three Abstractions in this Unit:

Computational problem:
- defines a computational task
- specifies what the input is and what the output should be

Algorithm:
- a step-by-step recipe to go from input to output (what your solution does)
- different from implementation

Correctness and complexity analysis:
- a formal proof that the algorithm solves the problem (why is what your solution does correct)
- analytical bound on the resources it uses

Pseudocode

- Problem: Given three numbers, find the largest.

- Algorithm (in plain English):
    1. Compare the first and second numbers.
    2. Note the larger of the two.
    3. Compare this number with the third.
    4. Note the largest number.
    5. Output the result.

Pseudocode

Plain English:

1. Compare the first and second numbers
2. Note the larger of the two
3. Compare this number with the third
4. Note the largest number
5. Output the result

Pseudocode – 3 numbers

```
BEGIN
 INPUT num1, num2, num3

 IF num1 > num2 THEN
       largest ← num1
 ELSE
       largest ← num2
 ENDIF

 IF largest < num3 THEN
       largest ← num3
 ENDIF

 OUTPUT largest
END
```

Pseudocode

- **Plain English:**

1. Denote the first number as the largest.
2. Compare the first and second number.
3. Note the larger of the two.
4. Compare this number with the third.
5. Note the largest number.
6. Continue for all the numbers
7. Note the largest number.
8. Output the result.

Pseudocode – n number of inputs

```
BEGIN
 INPUT n // Number of inputs
 INPUT numbers[1…n] // Array to
                       store the inputs

 largest ← numbers[1] // Initialise
                       largest with the
                       first number

 FOR i ← 2 TO n DO
       IF numbers[i] > largest THEN
              largest ← numbers[i]
       ENDIF
 ENDFOR
 OUTPUT largest
END
```

Some more examples!

Let’s Menti!

https://www.menti.com/alryd88kd5bn

Let’s take a break!

Kenny G – Havana

https://www.youtube.com/watch?v=N76txbrkDhE&pp=ygUHa2VuZWppINIHCQmiCgGHKiGM7w%3D%3D

Data Structures

Data Structure?

- A data structure is a way of organizing and storing data so that it can be accessed and modified efficiently.

- Purpose
    - To manage data in a way that suits the problem being solved.
    - To improve the efficiency of algorithms.

– **Analogy: Think of data structures as containers (e.g., boxes, shelves) designed for specific types of data.**

Why are Data Structures Important

- Enable efficient data manipulation

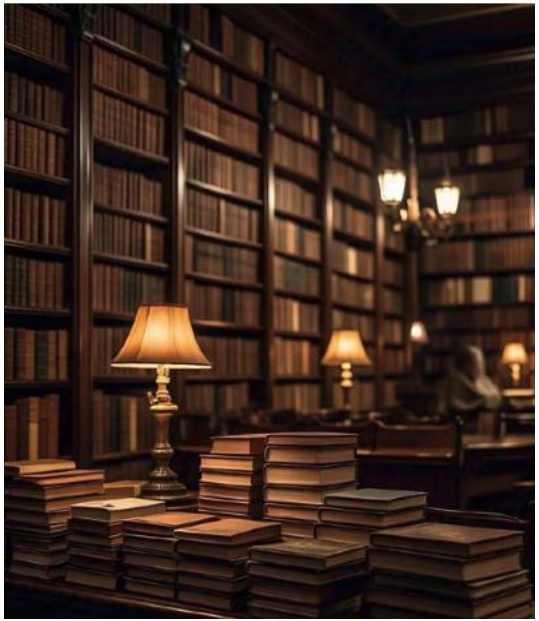
- Form the foundation of programming and software development

- Key to solving real-world problems like:
    - Storing customer records in a database
    - Managing tasks in a to-do list
    - Searching and sorting data in a system

A Real-world Example

- Problem: How do you organise books in a library?

- Possible Data Structures:
    - A list to keep all book titles
    - A dictionary to store books with their genres
    - A queue to handle books borrowing requests



Common Data Structures

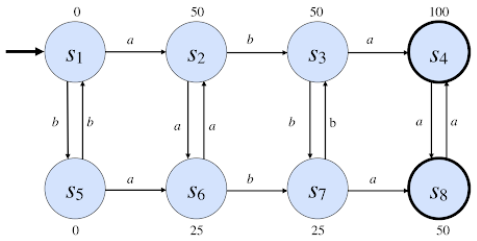
- Arrays
  - A **fixed** sized collection of elements stored in a contiguous memory locations, allowing efficient indexing and access
  - Example: numbers = [1, 2, 3, 4]

- List:
  - A **dynamic** sized ordered collection of elements
  - Example: numbers = [1, 2, 3, 4]

- Trees
  - A hierarchical data structure consisting of nodes, where each node has a value and links to child nodes
  - Example: binary tree

```
      10
     / \
    5   15
   /\  / \
  2  7 12  18
```

Common Data Structures



- Graphs
  - A collection of **nodes** (**vertices**) connected by **edges**.
  - Graphs can be **directed** or **undirected** and are use to represent relationships
  - Example: Graph as an adjacency list
    - graph = {
      ‘A’: [‘B’, ‘C’],
      ‘B’: [‘A’, ‘D’],
      ‘C’: [‘A’, ‘D’],
      ‘D’: [‘B’, ‘D’]
      }

Python

Introduction to Python

A high-level, interpreted programming language known for its readability and simplicity.

Data Types: Different kinds of data that Python can handle, such as:

Category	Data Type	Example
Text Type	str	x = “Hello World”
Numeric Type	int	x = 20
	float	x = 20.5
	complex	x = 1j
Sequence Type	list	x = [”apple”, “banana”, “cherry”]
	tuple	x = (”apple”, “banana”, “cherry”)
	range	x = range(6)

Introduction to Python

Category	Data Type	Example
Mapping Type	dict	x = {“name” : “John”, “age” : 36}
	set	x = {“apple”, “banana”, “cherry”}
	frozenset	x = frozenset({“apple”, “banana”, “cherry”})
Boolean Type	bool	x = True
Binary Type	bytes	x = b”Hello”
	bytearray	x = bytearray(5)
	memoryview	x = memoryview(bytes(5))
None Type	NoneType	x = None

Common Python Structures

- List: An ordered collection of items.
    - Example: my_list = [1, 2, 3, 4]

- Tuple: An ordered, immutable collection of items.
    - Example: my_tuple = (1, 2, 3)

- Dictionary: A collection of key-value pairs.
    - Example: {“Armin”: “COMP9123”, “Karlos”: “COMP9120”)
    - Example: {“name”: “Alice”, “age”:25}

Control Flows: ‘if’ Statement

Used to make decisions in code. Example:

```
if x > 0:
        print(“Positive”)
else:
        print(“Non-positive”)
```

Control Flows: Loops

- ‘**for**’ Loop: Iterates over a sequence

```
for i in range(5):
      print(i)
```

- ‘**While**’ Loop: Repeats as long as condition is true.

```
while x < 5:
      print(x)
      x += 1
```

Functions

```
def greet(name):
      return f”Hello, {name}!”

greet(“Alice”)

==============================

def add_numbers(a, b):
     return a + b

dd_numbers(5, 17)
```

Python Classes and Objects

- A Class is like an object constructor, or a “blueprint” for creating objects.
- Example:
    - Create a class named MyClass, with a property named x:

```
Class MyClass:
      x = 5
```

    - Create an object named p1, and print value of x:

```
p1 = MyClass()
print(p1.x)
```

Time & Space Complexity

The Big-O

- Big-O notation is a way to describe how the performance of an algorithm changes as the size of the input data grows.

- In simpler terms, it helps us understand how much time or space (memory) an algorithm needs as input get bigger

Why Big-O?

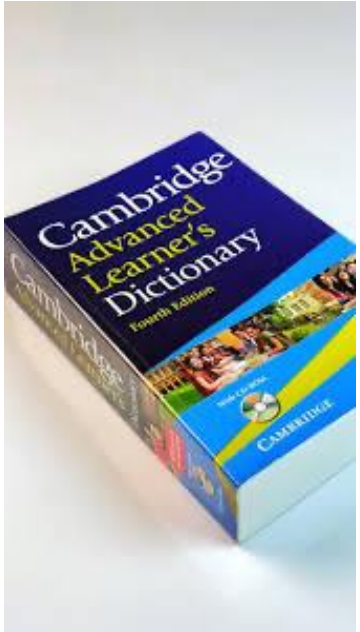
- Introduced to help **measure and compare** the efficiency of different algorithms.

- As data grows, some algorithms slow down more than others, and without Big-O, it would be difficult to compare algorithms and make informed decision when building efficient software

- It allows developers to:
  - Choose the **best** algorithm for a problem
  - Predict **how performance changes** with larger inputs
  - Optimise code to run **faster** and **use less memory**

A Real-world Example

- How do you find a word in a dictionary?



Time Complexity: Finding a Book in a Pile

- Linear Search (**O(n)**):
    - You start from the top and check each book one by one until you find the book you're looking for
    - If there are 10 books, you might need to look at up to 10 books to find the right one
    - If there are 100 books, you might need to look at up to 100 books
    - Big-O notation: **O(n)** where n is the number of books. The time it takes grows linearly with the number of books.

Time Complexity: Finding a Book in a Pile

- Binary Search (**O(log n)**) (assuming the books are sorted alphabetically)
    - You open the pile in the middle and check if the book is on the left side or the right side
    - Then you take relevant half and repeat the process
    - This way, you significantly reduce the number of books you need to check each time
    - Big-O notation: **O(log n)**. The time it takes grows logarithmically, which is much faster for large numbers of books.

Time Complexity: Finding a Book in a Pile

- For 10 books:
  - Linear Search: Check up to **10** books
  - Binary Search: Check up to **4** books (**$\log_2(10)$**)

- For 100 books:
  - Linear Search: Check up to **100** books
  - Binary Search: Check up to **7** books (**$\log_2(100)$**)

Space Complexity: Finding a Book in a Pile

- Measures how the memory usage of an algorithm increases with the size of input

- Linear Search (**O(n) Time, O(1) Space**):
  - You look at each book one by one
  - You don’t need any extra storage apart from the books themselves and a note where you can write down the current book you are checking
  - Space complexity: **O(1)** because the extra memory used does not grow with the number of books

- Binary Search (**O(log n) Time, O(1) Space**):
  - You keep splitting the pile of books in half until you find the right one.
  - You also do not need any extra storage aside from the books and maybe a note of the current section.
  - Space complexity: **O(1)** because, again, the extra memory does not grow with the number of books

Big O Examples (Homework)

- Big O (O()) describes the upper bound of complexity
- Omega (Ω()) describe the lower bound of the complexity

- Basic Operations:
  - Single arithmetic operations (e.g., +, *)
  - Single assignment (e.g. x:= 0)
  - Single test (e.g. x == 0)
  - Single read (of a primitive type: integer, float, character, bool)
  - Single write(of a primitive type: interger, float, character, bool)

Big O Examples (Homework)

(1) // Simple Assignment

```
x = 0 // O(1)
```

(2) // Series of Statements

```
x = 0 // O(1)
y = 2 // O(1)
z = x + y // O(1)
```

(3) // If Conditionals

```
if (cond) then
    block 1 // O(1)
else
    block 2 // O(1)
end if;
```

(4) // If Conditionals

```
if (cond) then
    block 1 // O(1)
else
    block 2 // O(n)
end if;
```

Big O Examples (Homework)

(5) // Loop

```
for x in 1…N:
    statements // O(1)
end
```

(6) // Complex Loop

```
for x in 1…N:
    statements // O(n)
end
```

(7) // Nested Loop

```
for x in 1…N:
    for y in 1…M:
        statements // O(1)
    end
end
```

(7) // Complex Nested Loop

```
for x in 1…N:
    for y in 1…M:
        statements // O(n^2)
    end
end
```

Big O Examples (Homework)

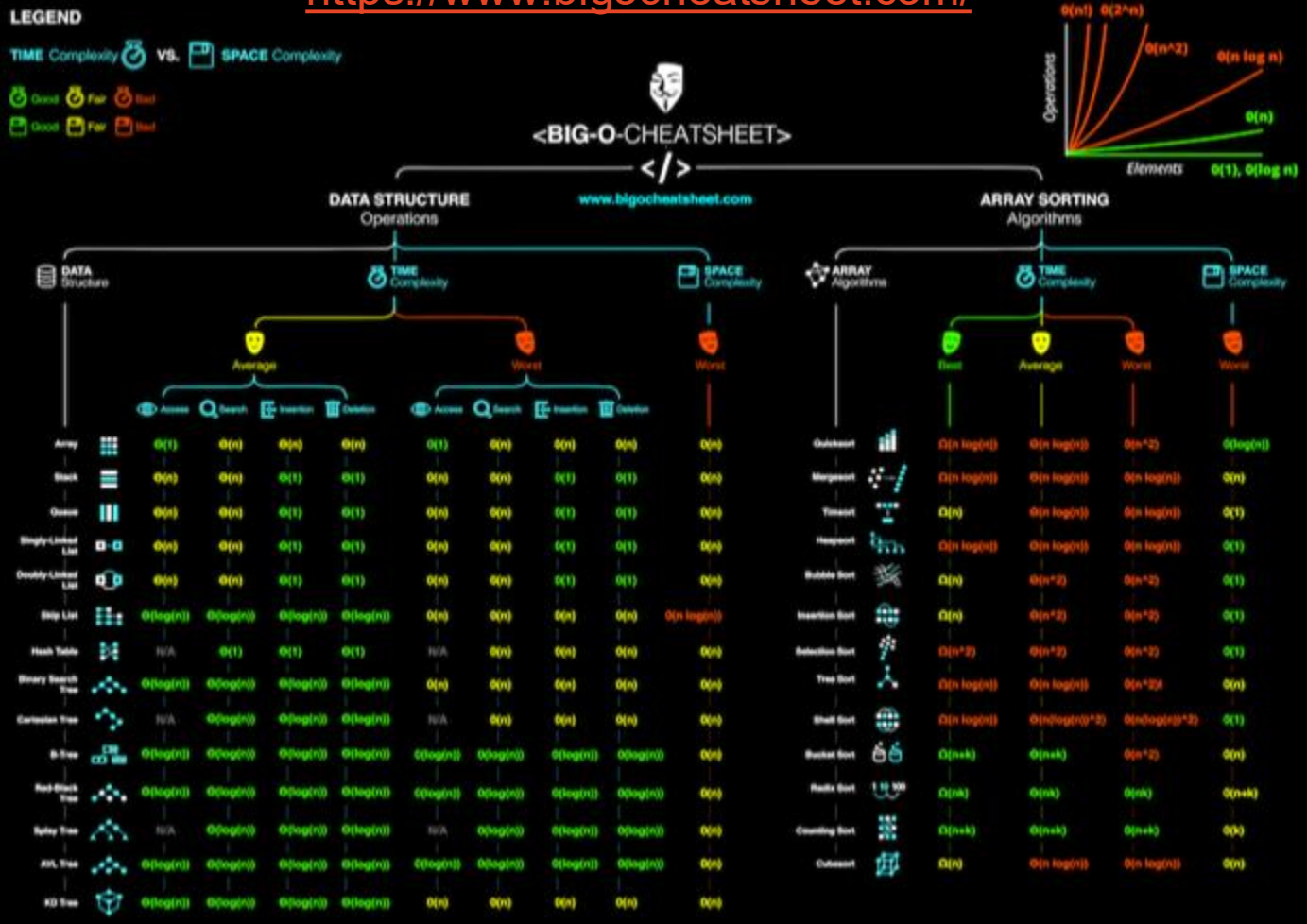
(9) //Binary Search def

```
binary_search(dictionary, word):
    low, high = 0, len(dictionary) - 1
    while low <= high:
        mid = (low + high)
        if dictionary[mid] ==word:
            return f"Found: {word}"
        elif dictionary[mid] < word:
            low = mid + 1
        else:
            high = mid - 1
    return "Not Found"
```

(10) //Managinig a Warehouse

```
for shelf in range(n):
    for row in range(n):
        for column in range(n):
            total_items +=
             warehouse[shelf][row][column]
return total_items
```

https://www.bigocheatsheet.com/



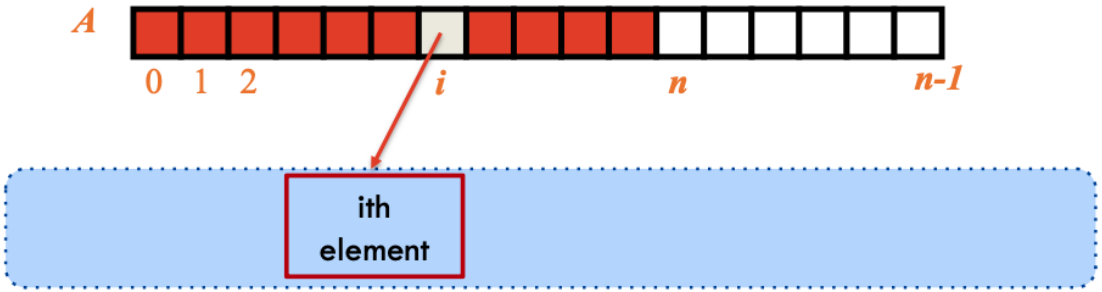
Arrays

What is an Array?

- A collection of elements stored in a fixed order

- Each element is placed in a specific position, called an index, which makes it easy to access and manage data

- Arrays are useful because they:
  - Store multiple values in a single structure
  - Allow quick access to elements using an index
  - Maintain a fixed order of element

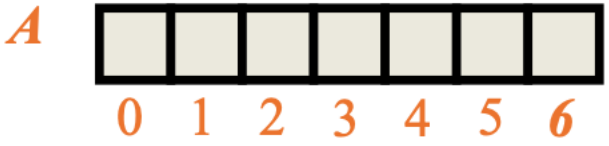


Declaration of an Array

- An array must be declared before being used

- Declaring an array means specifying the following:
  - Data type -- the kind of values it can store
    - Example: int, str, float
  - Name – to identify the array
  - Size – the maximum number of values that the array can hold

- Example: Create an int array A with size of 7



Array Operations

- Arrays support two basic operations that help manage and manipulate data efficiently.
- Set: Set an element at a given index
    - Example: Changing value 4 to 8

- Get: Retrieving an element using its index
    - Example: Accessing index 2 gives value 3

- Since arrays use contiguous memory locations, both get and set operations are extremely fast and work in constant time O(1), independent of the size of array (**n**).

Python Arrays?

- Python does not have built-in support for traditional arrays like some other programming languages

- Instead, it provide alternatives such as Lists which is a **flexible**, **dynamic collection** that can store **different data types**.

Pros and Cons of Array

- Pros:
  - **Fast Access:** Elements can be accessed instantly using an index
  - **Easy Iteration:** Supports simple loops for processing elements.

- Cons:
  - **Fixed Size:** the size of an array is usually predefined and cannot be changed dynamically
  - **Contiguous Block of Memory:** There must be enough free memory available in sequence, which can be challenging, especially when dealing with large arrays or fragmented memory
  - **Same Data Type:** Traditionally arrays store only one type of data (not an issue with dynamic lists like Python's).

Questions?

Fully anonymous feedback form for COMP9123 students – Fill out form

Updates

1.1 Tutors name are corrected. Spelling error. Thanks to Lindsey Deryckere for discovering it.

1.1 Big O Exampes Homework had a typo. thanks to Tianhong Zheng for noticing it.